CS 8803-O08: Project Phase 2

Santosh Pande
santosh.pande@cc.gatech.edu

Georgia Institute of Technology

Overview

The purpose of Phase 2 is to complete the front end of the Tiger compiler. You should use the code you completed for Phase 1 as the starting point. First, you will design and populate a symbol table. Next, you will utilize the symbol table to implement semantic checking. Finally, you will generate an intermediate representation (IR) code to be used as input for the back end of the compiler. To complete these tasks, you will utilize the parse tree generated in Phase 1 and the walking mechanisms provided by ANTLR4. For more information, we recommend you review:

- Definitive ANTLR4 Reference Manual
- CS 8803-O08: Tiger Language Specification
- CS 8803-O08: Intermediate Representation Specification
- Relevant textbook content: Ch4 - 4.2, Ch5 - 5.3.3, 5.4.1, 5.5
- Relevant lectures: P2L1, P2L2

1 Symbol Table

1.1 Implementation

The first step of Phase 2 is to design and implement a symbol table. The symbol table is the key data structure populated and used during compilation. Semantic checking makes extensive use of the symbol table. It holds declarative information about the constructs that make up a program. The following are typical (not exhaustive) entries in a symbol table:

- defined variables
- defined constants
- defined types
- function names and parameters
- literal constants
- source text labels
- compiler-generated temporaries

Each of these entries have multiple attributes associated with them. Typical attributes include:

- name
- type
- declaring procedure
- scope
- storage class
- for functions - number and type of arguments
- for arrays - dimension information

You have flexibility in the implementation details of the symbol table, and the above lists are not requirements. You should use the guiding principle of utility to govern how you construct your symbol table and what information it contains. Store what you will need for the next step of compilation. Your implementation will likely make use of one or more hash tables. You can find more information in the textbook, chapter 5.5.

Requirement 1

Implement a symbol table.

Requirement 2

When the `-t` flag is provided, write the symbol table to a file. The output file should have the same name and path as the input file with the extension changed to `.st`

1.2 Symbol Table File Format

The output format of your symbol table can be of your own design, but must include all relevant type and scope information in a structured form showing levels of nesting. Empty scopes, if encountered, may be included to show nesting levels. Use the following format as an example:

Command Line

```
$ cat example.st
scope 1:
    intArray, type, array, 10
    a, static, int
    b, static, intArray
    f1, func, null
    main, func, int
    scope 2:
        c, var, int
        scope 3:
            d, var, int
            e, var, int
            _t1, var, int
    scope 4:
        scope 5:
            f, var, int
            _t2, var, int
            _t3, var, int
```

In this format, the first line will always be the scope number followed by all the symbols in this scope and its nested scopes. Each line will contain the symbol name, symbol type, and base type separated by commas. In this example, scope 3 contains three symbols: d, e, and _t1, all `vars` with base type `int`. Scope 3 is nested in scope 2 while scope 4 is parallel to scope 2. Function symbols may not have a return value. Thus, the return type could be null.

1.3 Scoping

Scoping is a key aspect of symbol tables. The scoping rule we are going to use is the same rule used in most block structured languages: inner declaration of a variable name hides the outer declaration. In a given scope, the innermost declaration is the one that is visible and a name used in that scope is bound to that declaration. References always apply to the most closly nested block.

Your symbol table will track which symbols are defined in each scope. Additions to the symbol table do not overwrite previous declarations but instead mask them. Subsequent lookup operations that you run against a symbol return the most-recently inserted variable definition. A lookup operation on the symbol table should never fail. If a lookup fails, then the symbol table has not seen a declaration for the reference you are attempting to query.



Warning: It is a semantic error to use an undeclared symbol or to redefine a symbol in the same scope.

As you consider scoping, you are free to implement your symbol table using a series of nested scopes, or by using a global symbol table in which declarations are added upon entering a scope and removed upon exiting.

More details about scoping rules are defined in the Tiger language specification.

Requirement 3

Implement scoping according to the rules listed in the Tiger specification.

2 Semantic Checking

The second step of Phase 2 is to implement semantic checking. Implement semantic checking by walking the parse tree and using either the listener and visitor interfaces provided by ANTLR, or a mechanism of your own design. The semantic specifications for Tiger are detailed in the language specification. We recommend that you follow the approach below to implement the semantic checks:

1. Read and understand the semantic specification in the Tiger language specification.
2. Implement binding analysis to determine which variable name binds to which symbol table entry.
3. Check for use before declaration and redeclaration in the same scope.
4. Check for type mismatch. This includes binary operations, function return values, function parameters, and loop variables.
5. Check for additional errors using the provided test files as a guide.

Requirement 4

Implement semantic checking according to the rules listed in the Tiger specification.

When your compiler finds a semantic error, it must output a correctly formatted error message to standard error. You should follow the same error format as the auto-generated ANTLR lexer and parser. The required format is:

```
line line_number:column_number description
```

Where line_number is replaced by the actual line number, column_number is either blank or replaced by the actual column number, and description is a descriptive field describing the problem. Your compiler should report as many errors as it can find before exiting.

Requirement 5

Print semantic errors to the **standard error**.

Requirement 6

If your compiler encounters one or more semantic errors, exit with code `4`.

3 Intermediate Representation

The final step of Phase 2 is to convert the program into IR. For this project, we will be using 3-address code to represent the IR. 3-address code is often called quadruples due to the quad format: opcode, src1, src2, dst. The IR Specification defines the syntax of these quadruples and the available operations. After generating the IR, you will output the instruction stream to a file. The file format is defined in the IR Specification as well.

Requirement 7

Generate 3-address IR for the supplied valid Tiger program.

Requirement 8

When the `-i` flag is provided, output the IR to a file using the format defined in the IR specification. The IR output file should have the same name and path as the input file with the extension changed to `.ir`. Do not generate an IR file for programs containing lexical, syntactic, or semantic errors.

4 Report

Write a concise report detailing the design of your compiler front end. Your report must include, but is not limited to, the following information:

- Design of your symbol table
- Description of method of semantic checking including any listeners/visitors used
- Description of method of IR generation including any listeners/visitors used
- Bugs or features that are not fully operational

Requirement 9

Write design internals report. The report should be named `phase2_design.pdf`.

5 Grading

Your project should be submitted to Gradescope for grading. It is important that your project follows the correct build process and naming conventions outlined below. Phase 2 is an extension of Phase 1. Therefore, the deliverables for both phases should be included.

5.1 Building

The build environment is the same as in Phase 1. In order to facilitate the building and testing of your code, you must include a *Makefile* at the root of your source code directory. Your *Makefile* should generate the lexer and parser code, then compile and build a single output file under the `cs8803_bin` folder. If using Java, the output file should be a JAR file called `tigerc.jar`. If using C++, the output file should be an executable file called `tigerc`.

5.2 Testing

We will test your symbol table, semantic analysis, and IR generation on multiple test inputs. Some of the test cases will be provided to you before the due date.

5.2.1 Test Environment

The test environment is the same as in Phase 1. The environment we use to test your code is based on vanilla Ubuntu 22.04 with ANTLR4 libraries installed. You can build a local test environment using Vagrant or Docker from the provided setup scripts. The test environment provides a convenient platform to verify your project builds and to directly run individual test cases while troubleshooting.

5.2.2 Gradescope

You can also test your compiler by submitting early to Gradescope. Gradescope will run **make** to generate the executable, and then run a series of automated tests. Note that additional points may be deducted for errors that are not caught by the automated testing. You can submit your compiler to Gradescope as many times as you would like prior to the deadline. It is highly recommended to **submit early and often**.

5.3 Timeline

Start early! Make sure to submit to the auto-grader early and often to check your code. Here is a suggested timeline showing important milestones:

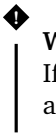
- Week 1 : Complete Symbol Table
- Week 2 : Complete Semantic Checking
- Week 3 : Complete IR generation

5.4 Deliverables

Your project should be submitted as a .zip file to Gradescope. The .zip file should contain:

- phase2_design.pdf - Design internals report
- Makefile - Makefile
- Tiger.g4 - ANTLR4 lexer/parser specification
- src - Directory containing your compiler source code (C++/Java)

5.5 Rubric



Warning: Your submission is expected to compile cleanly from source, **including ANTLR classes**. If a submission fails to compile, contains pre-built class files or binaries, or artificially bypasses the auto-grader, it can result in a **zero**.

Symbol table generation	**10 points**
Semantic analysis	**30 points**
IR generation and correctness	**50 points**
Report detailing the design internals	**10 points**